

Part 1

What a Transport Fever 3 vehicle actually is

A TF3 vehicle is a Lua table: a tree of named nodes pointing at meshes and materials, plus metadata structs that tell the simulation which nodes are axles, where people sit, how much it weighs and how fast it goes. The game does the driving.

In this part

1. The .mdl: model tree + metadata
2. The files around it
3. Node names are the contract (compared with TSC)
4. How the game puts a rail vehicle on the track
5. Events instead of control values
6. The simulation: four numbers, not a curve set
7. Passengers, cargo and seats
8. Locomotive, wagon, multiple unit
9. Load order and where scripts fit

Structures and field names are from the supplied Vehicle Basics, Vehicle Types and Advanced Topics pages.

1 • The .mdl: model tree + metadata

```
function data() return {  
  lods = {  
    { node = { name = "RootNode", children = {  
      { name = "body", mesh = "msh/body_lod0.msh",  
        materials = { "mat/body.mtl", "mat/glass.mtl" },  
        transf = { 1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1 },  
        animations = { open_doors_left = { ... } } },  
      { name = "bogie_front", children = {  
        { name = "w1", mesh = "msh/w1_lod0.msh" }, ... } },  
      ... } }, visibleFrom = 0, visibleTo = 150 },  
    { -- LOD 1: node tree again, _lod1 meshes  
  },  
  metadata = {  
    transportVehicle = { ... }, landVehicle = { ... },  
    railVehicle = { config = { axles = { "w1", "w2" }, fakeBogies = { {}, {} } } },  
    seatProvider = { ... }, soundConfig = { ... }, emissions = { ... },  
    availability = { ... }, description = { ... }, cost = { ... }, ... },  
} end
```

Model tree (per LOD)

Each node: a name, an optional mesh, materials (one per mesh group), a transform, children, animations.

The Model Editor writes all of this from your FBX. You rarely edit it.

Metadata

What makes the model a vehicle.
Edited in the Model Editor's Metadata window or seeded by an .import.lua.
Refers to nodes by name.

Figure 1.1 • Shape of a vehicle .mdl. Field names (lods , node , children , mesh , materials , transf , animations , metadata , visibleFrom/visibleTo) are those shown in the repaint and advanced-topics examples [manual](#); the exact nesting of lods[i].node is from those examples, the rest of the layout [check](#).

2 • The files around it

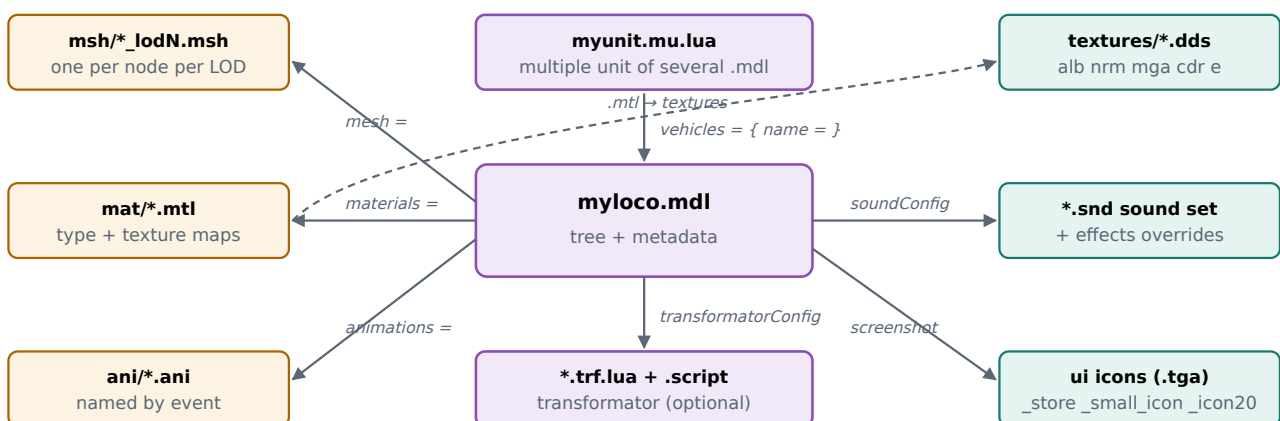


Figure 1.2 • The TF3 vehicle file graph. Compared with TSC's graph (TSC guide, Figure 1.1) there is no separate bogie file, no simulation file, no cab, and no per-vehicle script: the .mdl carries bogie data as node names and physics as a few numbers. [manual](#) for all references shown; folder names [check](#).

3 • Node names are the contract

“In comparison to Transport Fever 2, most of the metadata refer to model parts by using the node names. Hence it is important to set proper names for the nodes.” [manual](#) This is the same discipline the TSC guide calls “the glue”, but the links point in a different direction:

Link	TSC	TF3
Axle / wheel spins	Bogie blueprint <code>Axle[].NodeId</code> → IGS node; radius typed in.	<code>railVehicle.config.axles = { "w1", ... } → node</code> ; rotation about the node's own y axis; radius comes from the mesh. manual
Bogie turns	Geometry node + <code>BogeyPivotX</code> typed in.	Any node that <i>contains axle nodes</i> is a bogie; pivot = its mesh origin. Nothing typed. manual
Driver	<code>DriverPosition</code> matrix; cab camera.	<code>seatProvider.seats[]</code> with <code>group</code> = node name and a <code>transf</code> matrix; crew model placed by the game. manual
Doors / pantograph / wipers	Control value → <code>.ia</code> .	Node has an <code>animations</code> entry keyed by event name → <code>.ani</code> . manual
Lights	Light child entities + glow meshes switched by direction.	Light sources imported from FBX lights (point/spot/sun) with <code>childId</code> = parent node; headlight meshes shown/hidden by <code>front_forward_parts_on/off</code> events. manual
Smoke	Particle emitter child + Lua <code>SetEmitterRate</code> .	Locator named <code>...emitter...</code> ; <code>smoke_dark</code> / <code>smoke_bright</code> in the name sets defaults; transformator scales it with speed. manual
Cargo	-	<code>loadIndicator.cargoBay.childId</code> and <code>toHide</code> node lists. manual

Name rules the importer enforces

- Node name = the Blender **object** name; the `.msh` file name = the Blender **mesh datablock** name. [manual](#)
- Names may not contain `|`, `%` or `#`; lower-case words joined by `_` are the convention. The LOD suffix is added automatically. [manual](#)
- Everything after `#` is a comment and is dropped: use it to hide Blender's `.001` or to give two LODs' nodes the same name. [manual](#)
- `|...` after a name is a **tag**: material shortcode, `|seat_<anim>`, `|bounding_box`, `|extent`, `|collider_list`, `|label`, `|draft_animal_...`. [manual](#)
- A name starting with `/` or `modid::/` *references* an existing mesh or material instead of creating one. [manual](#)

4 · How the game puts a rail vehicle on the track

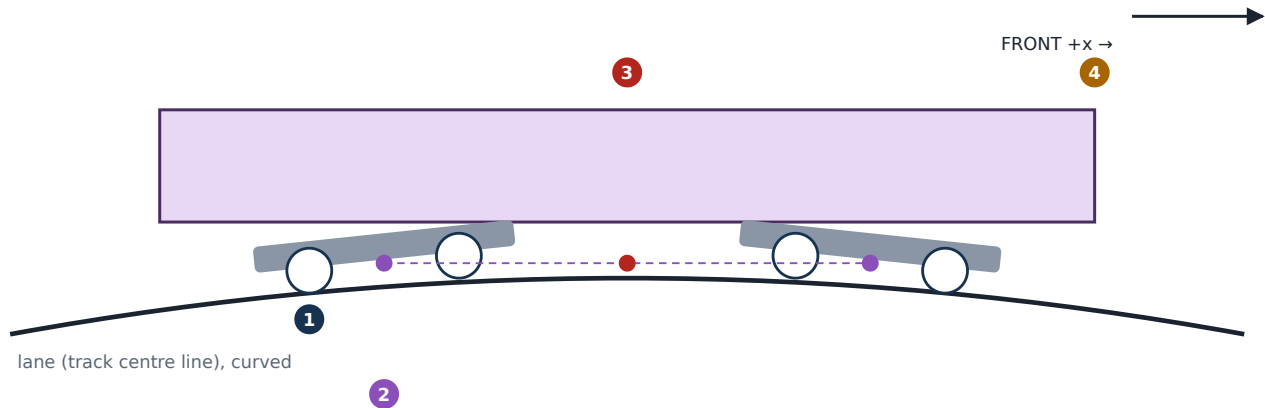


Figure 1.3 · Alignment rules [manual](#). **1** Each axle node's origin sits centred above the lane and the axle rotates about its y axis as the vehicle rolls. **2** A node that contains axle nodes is a bogie; it pivots at its own origin, which should sit midway between its axles. **3** A node that contains two bogies (the body) is aligned along the line between the two bogie pivots. Nodes with more than two axles/bogies use the outermost. **4** Where a part has no second bogie (articulated sections, single-truck cars) you add a **fake bogie** (Part 4) so the game has a second point to align to.

Consequence for the shared model: **the bogie pivot you model is the pivot the game uses.** There is no `BogeyPivotX` to disagree with the mesh. The TSC-side problem that the toolkit patches (bogie at 5.583 vs pivot 6.25) cannot occur in TF3. Conversely, a bogie origin that is “a bit off” in Blender is immediately visible in TF3 curves.

5 · Events instead of control values

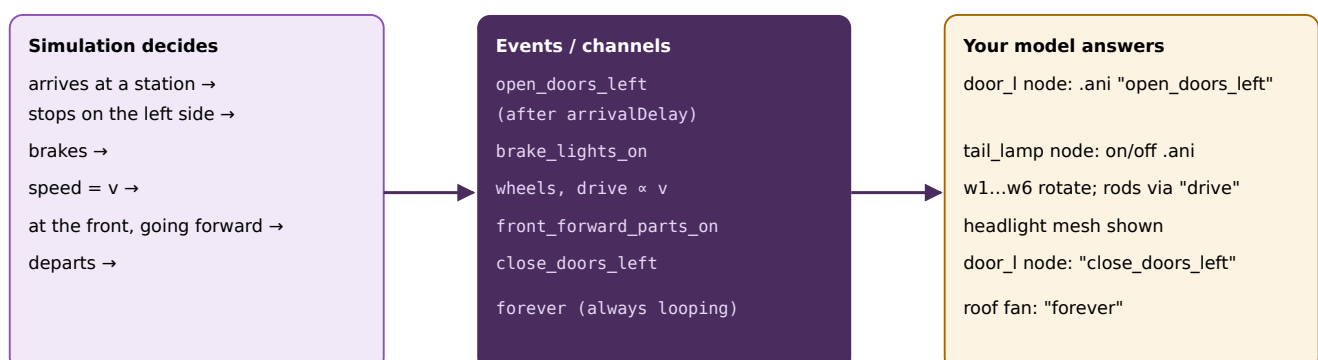


Figure 1.4 · The event model [manual](#). Where TSC has a bus of control values that the player, AI and Lua write, TF3 has a fixed vocabulary of events the simulation fires. A node that has an animation of that name reacts; a node that doesn't, ignores it. You cannot invent a new event without a transformator script (Part 5), but you can attach any animation to the existing ones.

6 · The simulation: four numbers, not a curve set

```
landVehicle = {
  engines = { { power = 150, tractiveEffort = 30, type = "ELECTRIC" } }, -- kW, kN;
HORSE/STEAM/DIESEL/ELECTRIC
  topSpeed = 19.444, -- m/s (70 km/h)
  weightEmpty = 14000, -- kg
  weightMaxPayload = 5000, -- kg added when fully loaded
  friction = 0.02,
  brakeDeceleration = 2.5, -- m/s2
},
```

manual A vehicle may have zero (wagon), one or more engines. There is no throttle curve, no adhesion model you author, no brake pipe: acceleration follows from power, tractive effort, mass and friction; deceleration is a constant. The TSC simulation blueprint's numbers map onto this directly:

TSC source	TF3 field	Conversion
Sim MaxPower (6600, label-less)	engines[].power kW	If the TSC value is kW, copy. If hp (7F: 1328 hp), × 0.7457.
Sim MaxForce (300)	engines[].tractiveEffort kN	If kN, copy. If klbf (7F: 35.296), × 4.448.
Sim MaxSpeed (145 mph) / bp MaxPermissibleSpeed	topSpeed m/s	mph × 0.44704; km/h ÷ 3.6.
Bp Mass (84 t)	weightEmpty kg	× 1000.
Subsystem class	engines[].type	Electric → ELECTRIC, DieselElectric → DIESEL, Steam → STEAM.
Bp RollingFrictionCoefficient	friction	Different model; start from TF3 defaults (0.02 shown) and tune. check
Brake blueprint	brakeDeceleration	No mapping; pick a plausible m/s ² (2.5 shown) and tune.

Balancing matters more than realism here: TF3 is a tycoon game, and the manual asks that strong vehicles carry higher noise/pollution (emissions.noise.score / pollution.score 0–100, or –1 for automatic). **manual**

7 · Passengers, cargo and seats

None of this exists in TSC; all of it is required in TF3 for the vehicle to be useful:

- transportVehicle.compartmentsList : compartments → loadConfigs → cargoEntry with capacity , cargoTypeSet (classes/types included or excluded), optional loadIndicator and seats ; toHide lists nodes hidden in that configuration. **manual**

- `seatProvider`: `drivingLicense` (RAIL...), optional `crewModels`, and `seats[]`, each with `animation` (`driving`, `driving_upright`, `idle`, `sitting`, `walk`), `crew`, `forward`, `group` (anchor node) and a 16-value `transf`. Seats can be placed in Blender as empties named `...|seat_<anim>` or `...|seat_crew_<anim>` and are imported. [manual](#)
- `loadSpeed` ($\approx$ number of door lanes, or -1), `comfortFactor`, `arrivalDelay` / `departureDelay` in ms, which should equal your door animation lengths. [manual](#)

8 · Locomotive, wagon, multiple unit

Kind	Folder	Distinguishing metadata
Locomotive	/vehicle/train	<code>landVehicle.engines</code> non-empty; <code>transportVehicle.engineTransportModes</code> set (e.g. <code>ELECTRIC_TRAIN</code>); usually a crew seat. manual
Wagon / coach	/vehicle/wagon	No engines; compartments define passengers or cargo; <code>transportModes</code> <code>TRAIN</code> (and <code>ELECTRIC_TRAIN</code> if it may run under wire). manual
Tram	/vehicle/tram	Carrier <code>TRAM</code> ; same events plus blinkers. manual
Multiple unit	<code>.mu.lua</code> next to the <code>.mdl</code> files	<code>vehicles = { { name = "x.mdl", forward = true }, ... }</code> ; the game sums the technical data. A vehicle with <code>multipleUnitOnly = true</code> is never sold alone. manual
Reversible train	-	First, last and every motorised vehicle have <code>transportVehicle.reversible = true</code> ; parts switch with the <code>..._forward/backward_parts_on/off</code> events. manual

The TSC “engine vs wagon” split (TSC guide Part 1 §7) therefore maps cleanly: the same two Blender models become a TF3 locomotive and a TF3 wagon, and the TSC “tender” becomes either part of a multiple unit or a wagon with no capacity.

9 · Load order and where scripts fit

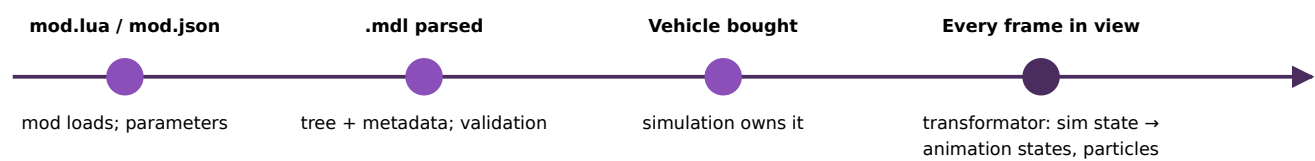


Figure 1.5 · Where code runs. The only per-vehicle code path is the transformator, and it runs only for rendering (“no access to api functions other than `api.type`”). [manual](#) Gameplay-changing logic lives in game scripts, which are outside this guide's scope.